Habib University
shaping futures

CS201 – Data Structures II

Lecture 09

Dr. Muhammad Mobeen Movania

(Some content has been taken from lecture slides of
Dr. Syeda Saleha Raza)

Outline

- Recap of Trees and Tree Terminologies
- Binary Trees
- Binary Search Trees (BST)
- Randomized Binary Search Trees
- Treaps
- Creation
- Operations on Treaps

Recap of Trees

- Linear list
  - each element can have one successor
  - Examples
    - Arrays, LinkedList, Doubly LinkedList, Stack, Queue, Deque

- Non-linear list
  - each element can have more than one successor
  - Examples
    - Tree
    - Graph
- Tree
  - an element can have only one predecessor
- Graph
  - an element can have one or more predecessors

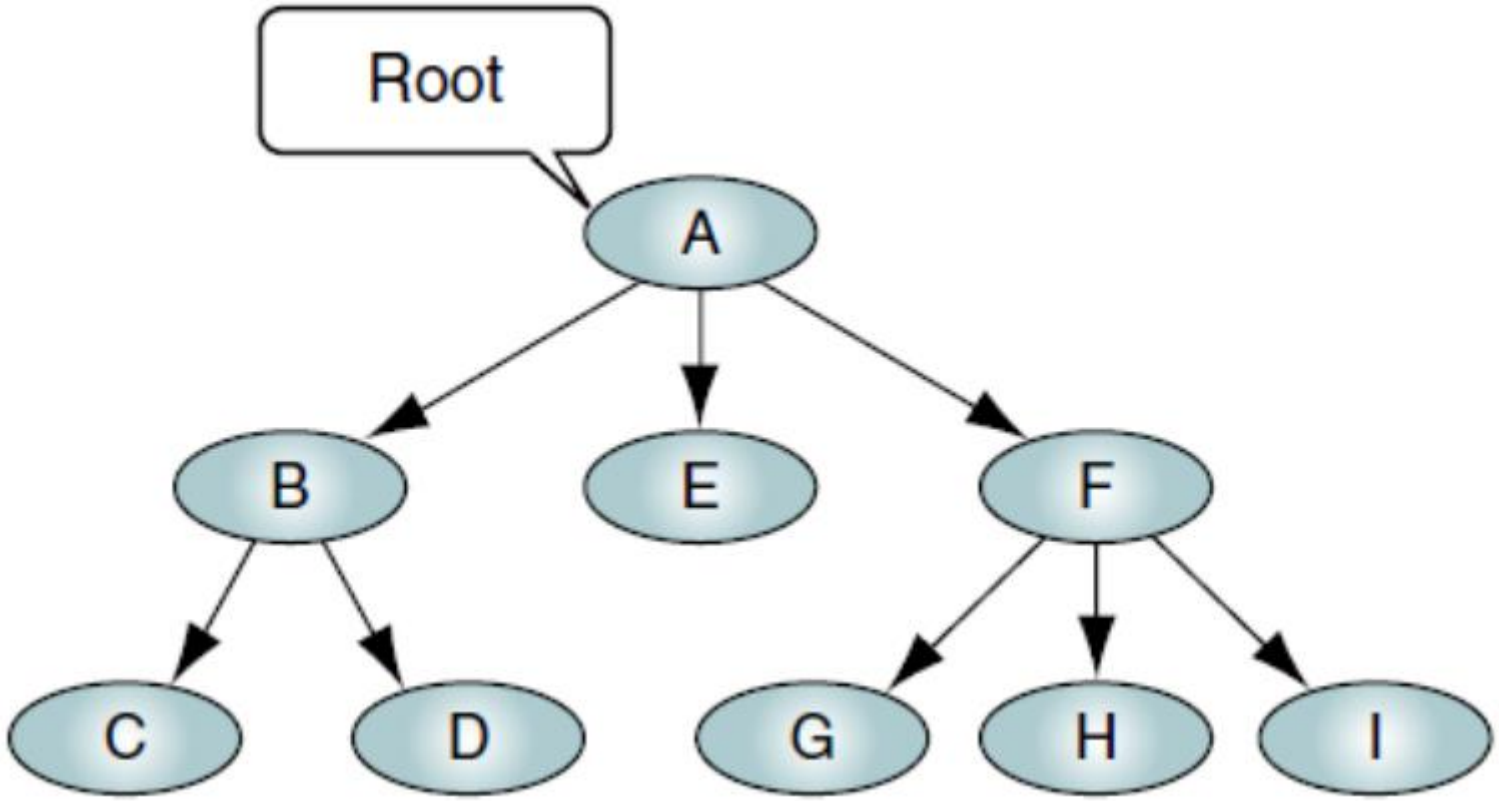
Tree and Tree Terminologies

- A tree consists of a finite set of elements, called **nodes**, and a finite set of directed lines, called **branches**, that connect the nodes
- The number of branches associated with a node is the **degree** of the node
- Two types of branches
  - When the branch is directed towards the node, it is an **indegree** branch
  - When the branch is directed away from the node, it is an **outdegree** branch
- The sum of the indegree and outdegree branches is the **degree** of the node

Tree and Tree Terminologies

- If the tree is not empty, the first node is called the **root**
  - The **indegree** of the root is, by definition, **zero**
- With the exception of the root, all of the nodes in a tree must have an indegree of exactly **one**; that is, they may have **only one predecessor**
- All nodes in the tree can have zero, one, or more branches leaving them; that is, they may have an **outdegree of zero, one, or more** (**zero or more successors**)
- Trees are used extensively in computer science
  - to represent algebraic formulas
  - as an efficient method for searching large, dynamic lists
  - for applications such as artificial intelligence systems and encoding algorithms.

Example of a Tree

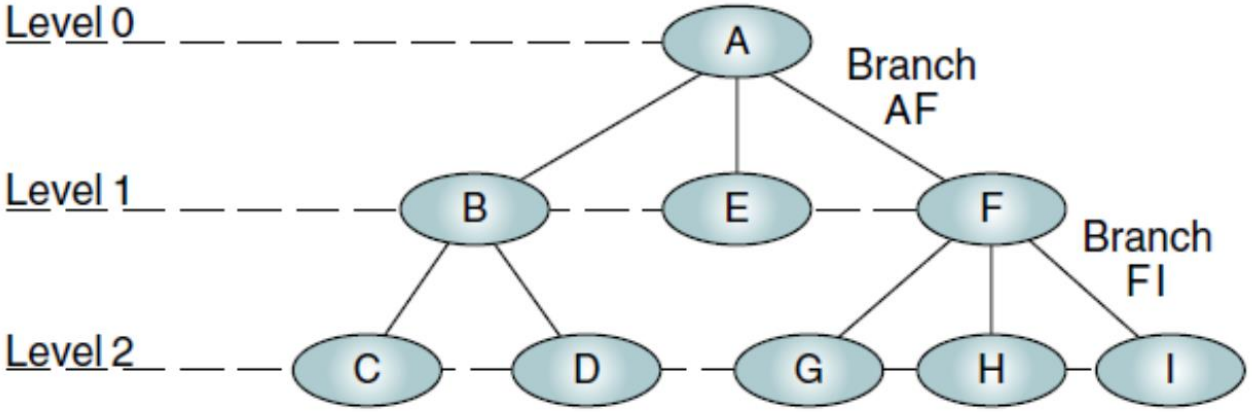


Tree Terminologies

- A path is a sequence of nodes in which each node is adjacent to the next one
- Every node in the tree can be reached by following a unique path starting from the root
- The level of a node is its distance from the root
  - As the root has a zero distance from itself, the root is at level 0
  - The children of the root are at level 1, their children are at level 2, and so forth
- The height of the tree is the length of the longest path from u to one of its descendants
  - By definition the height of an empty tree is -1
  - The depth of a node, u, in a binary tree is the length of the path from u to the root of the tree

Tree Terminologies

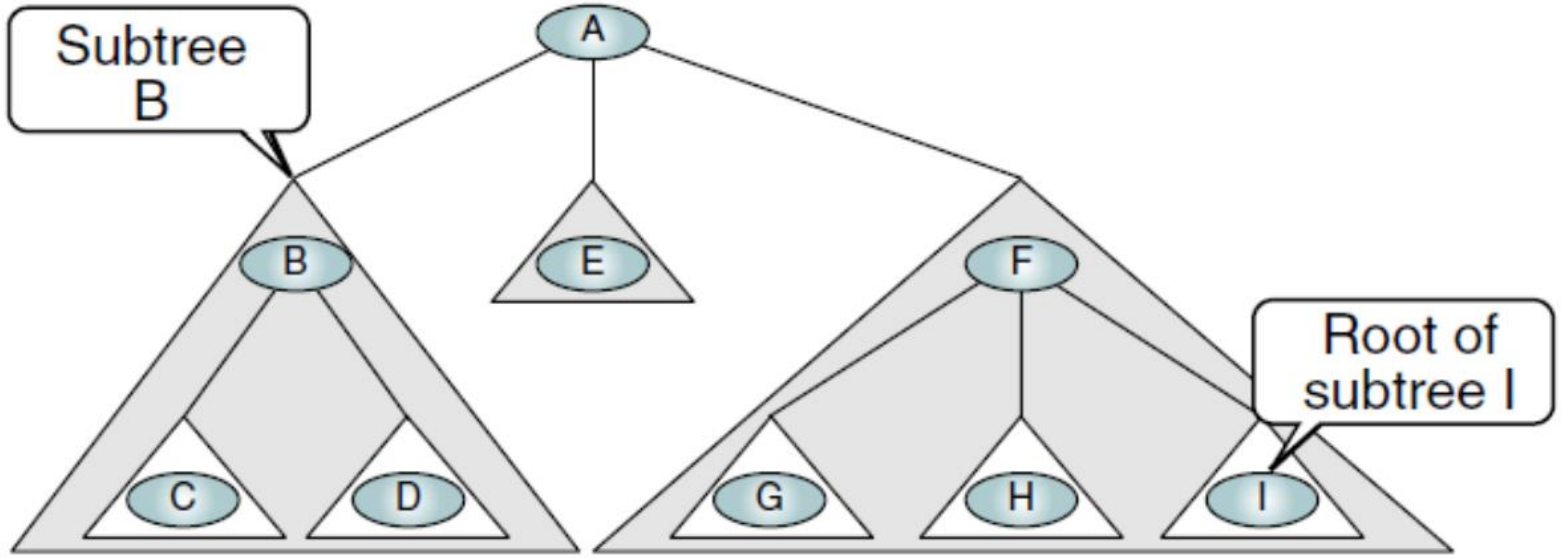
- Siblings are always at the same level, but all nodes in a level are not necessarily siblings
  - For example, at level 2, C and D are siblings, as are G, H, and I.
  - However, D and G are not siblings because they have different parents



Root: A	Siblings: {B,E,F}, {C,D}, {G,H,I}
Parents: A, B, F	Leaves: C,D,E,G,H,I
Children: B, E, F, C, D, G, H, I	Internal nodes: B,F

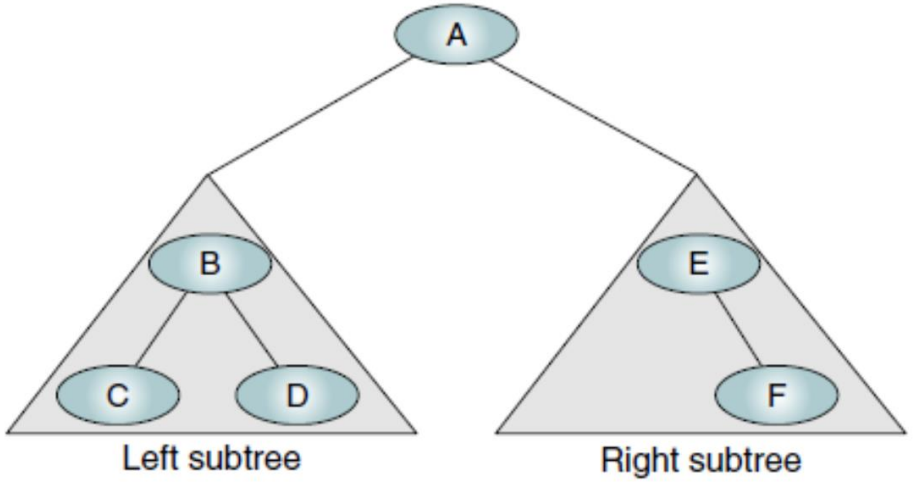
Tree Terminologies

- A tree may be divided into subtrees
  - A subtree is any connected structure below the root
  - The first node in a subtree is known as the root of the subtree and is used to name the subtree
  - Subtrees can also be further subdivided into subtrees

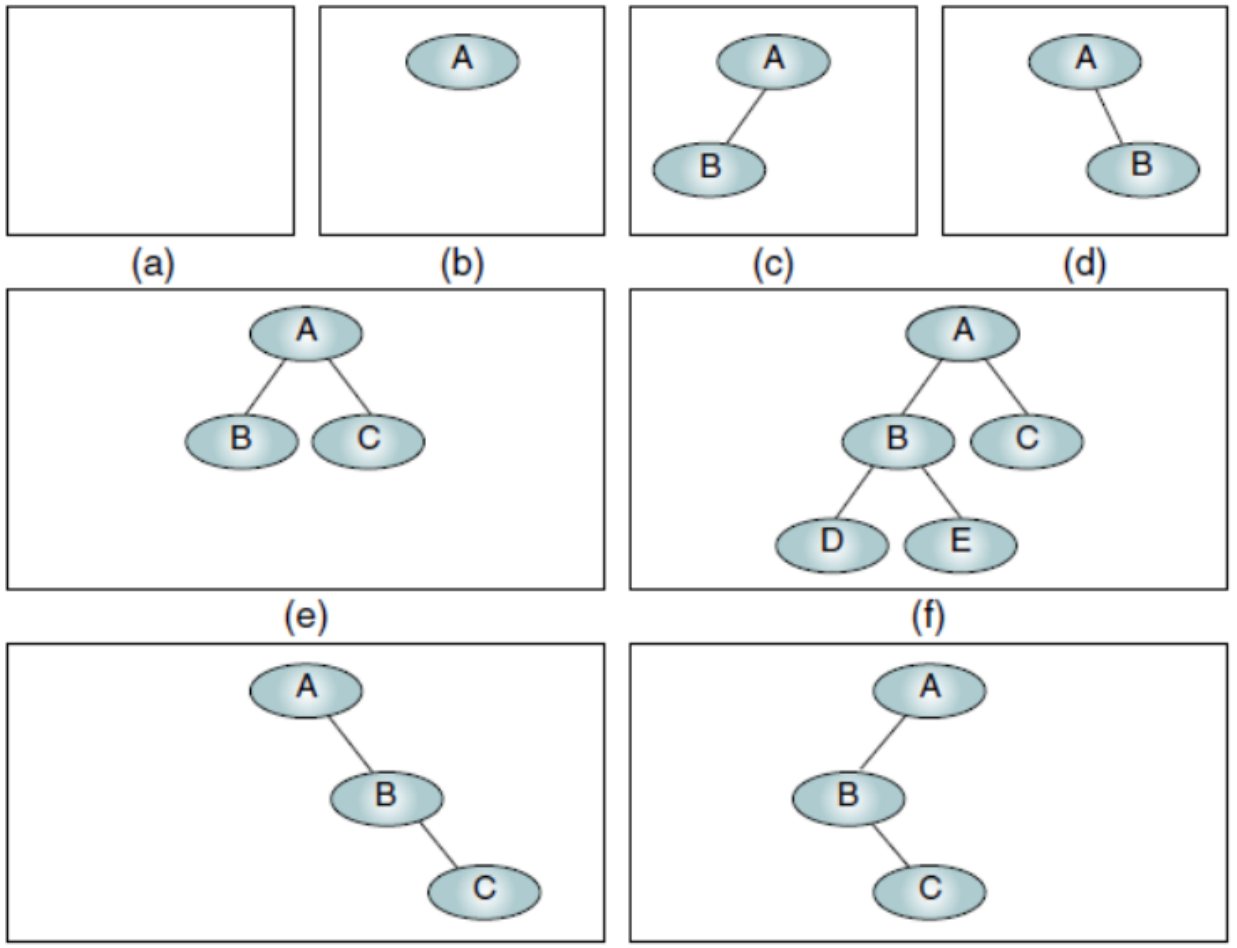


Binary Tree

- A binary tree is a tree in which
  - no node can have more than two subtrees
  - the maximum outdegree for a node is two
  - In other words, a node can have zero, one, or two subtrees
  - These subtrees are designated as the left subtree and the right subtree
- Symmetry is not a requirement of binary trees

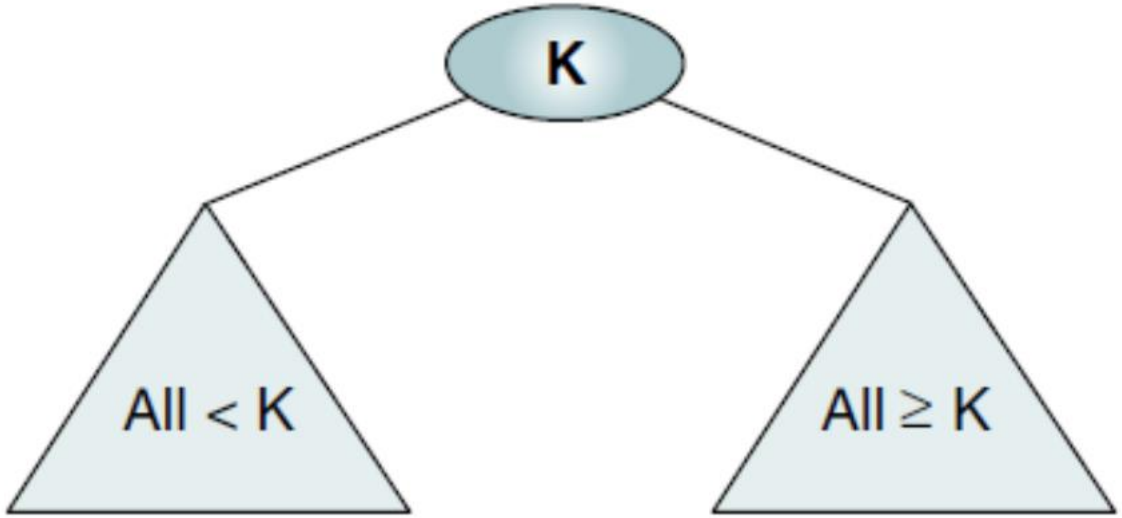


Examples of Binary Trees

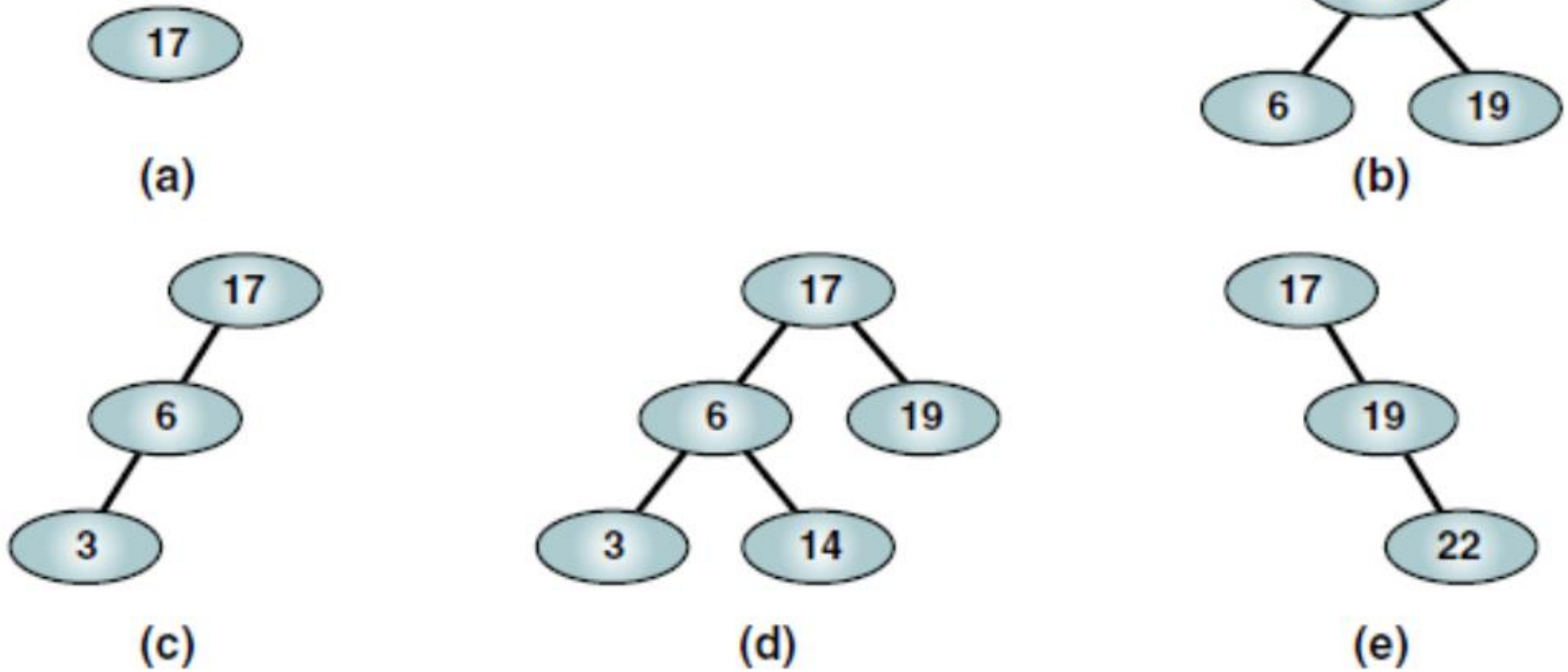


What is a Binary Search Tree (BST)?

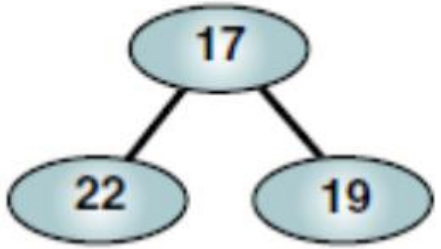
- BST is a binary tree with
  - All items in the left subtree are less than the root.
  - All items in the right subtree are greater than or equal to the root.
  - Each subtree is itself a binary search tree



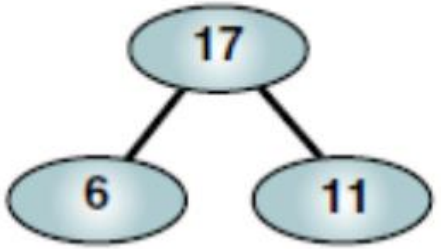
Examples of Valid BSTs



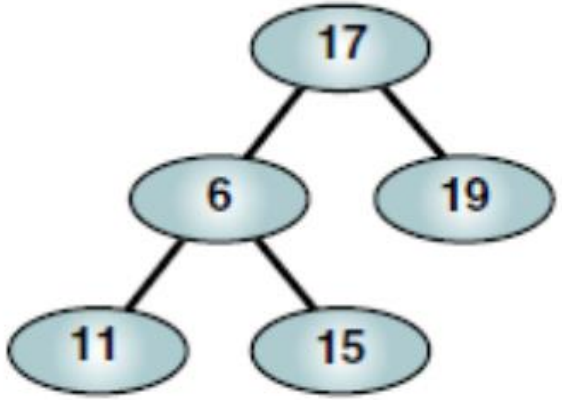
Examples of Invalid BSTs



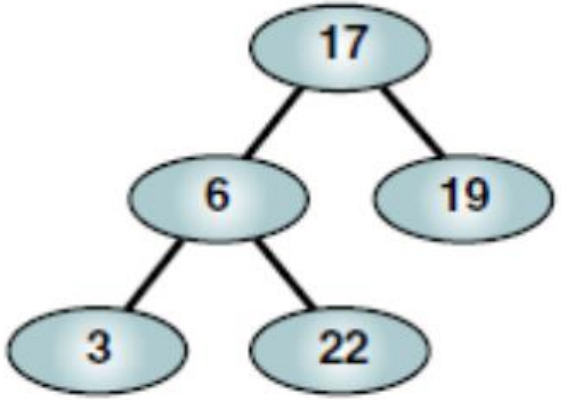
(a)



(b)



(c)



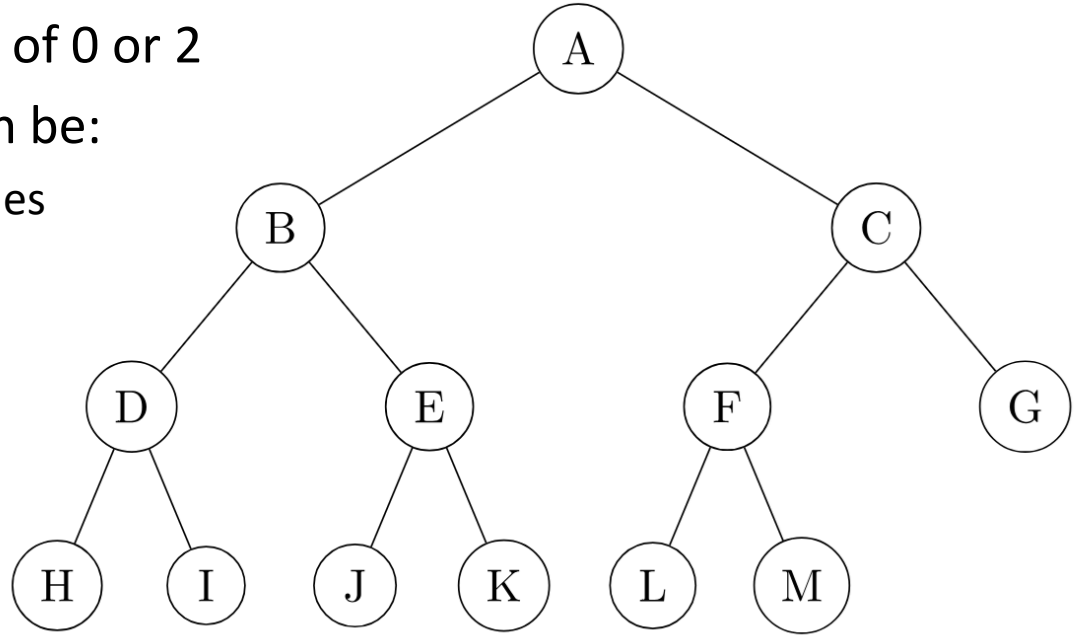
(d)

Types of Binary Trees

- **Full Binary Tree**: Every node has either 0 or 2 child nodes, i.e., left and right or no children
- **Complete Binary Tree**: All levels, except possibly the last, are filled, and all nodes are as left as possible
- **Perfect Binary Tree**: All nodes have exactly two children, and all leaf nodes are at the same level
- **Balanced Binary Tree**: The heights of any node's left and right subtrees differ by at most one
- **Degenerate Binary Tree**: Each parent node has only one child node

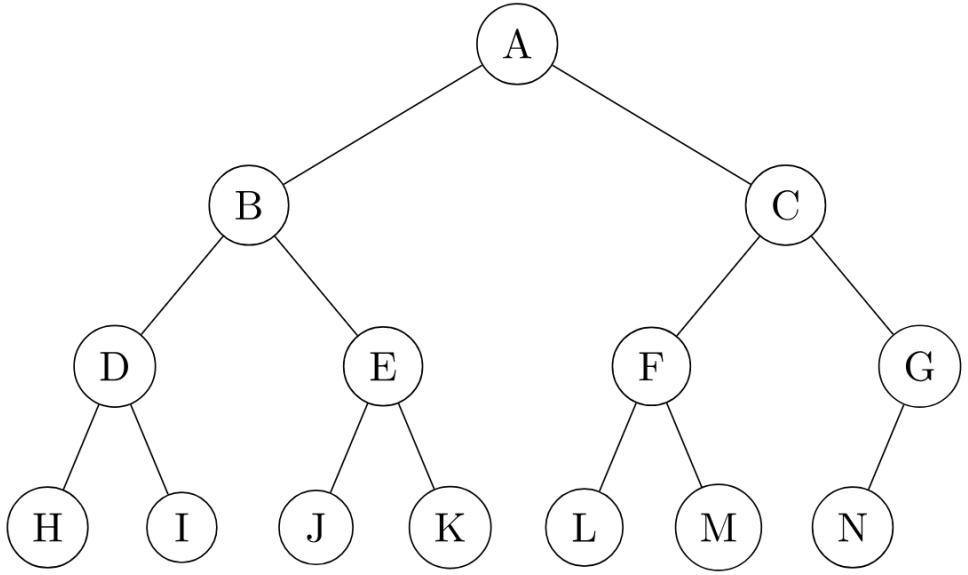
Full Binary Tree

- Useful properties
  - Always balanced
  - Height of full binary tree with levels k is k-1
  - Total number of nodes for full binary tree with height h are 2^{h+1}-1
  - Every node of a full binary tree can have a degree of 0 or 2
  - The total number of nodes in a full binary tree can be:
    - 2i+1, where i is the number of internal or non-leaf nodes
    - 2l-1, where l is the number of leaves



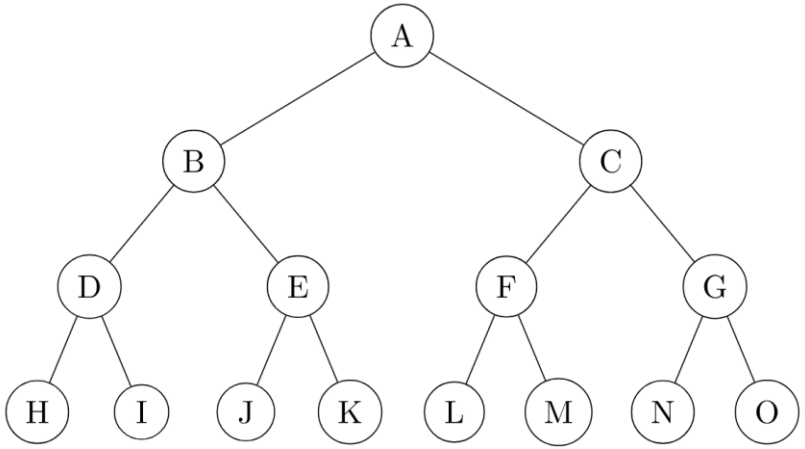
Complete Binary Tree

- Useful properties
  - The number of nodes at a depth d is at most 2^d
  - The height of a complete binary tree with n nodes is floor(log(n))
  - All leaf nodes in a complete binary tree are present in the last level or the penultimate level



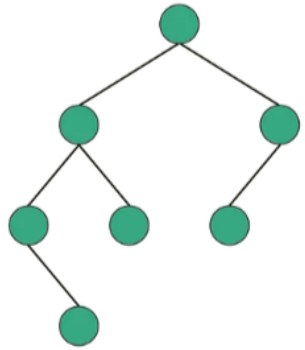
Perfect Binary Tree

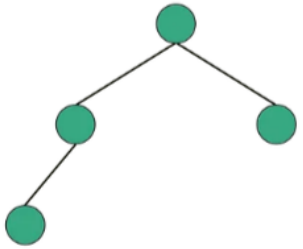
- Useful properties
  - Always symmetric and balanced
  - No. of leaf nodes in a perfect binary tree with n internal nodes is n+1
  - No. of nodes in a perfect binary tree with height h is $2^{(h+1)} - 1$
  - All internal nodes have a degree of 2
  - All leaf nodes have a degree of 0
  - The height of a perfect binary tree with N nodes is $\log_2(N + 1)-1$

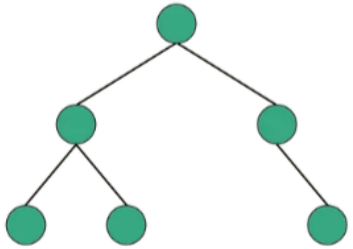


Balanced Binary Tree

- Binary tree in which height of the left and the right sub-trees of every node may differ by at most 1.

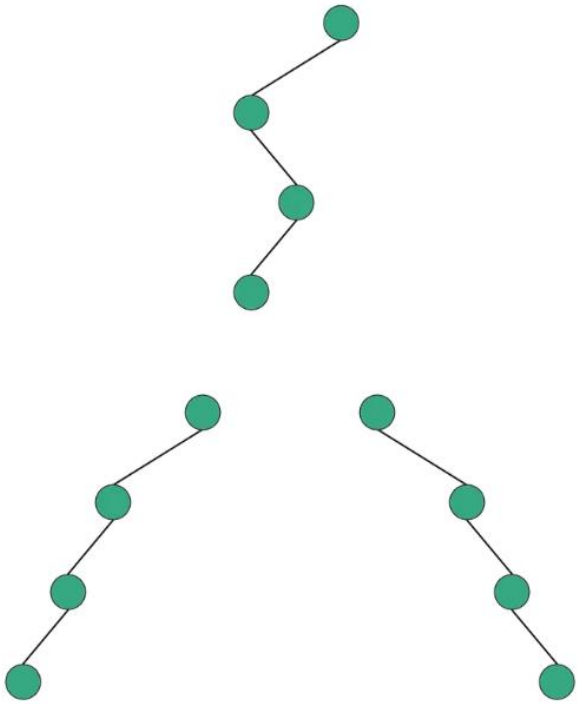


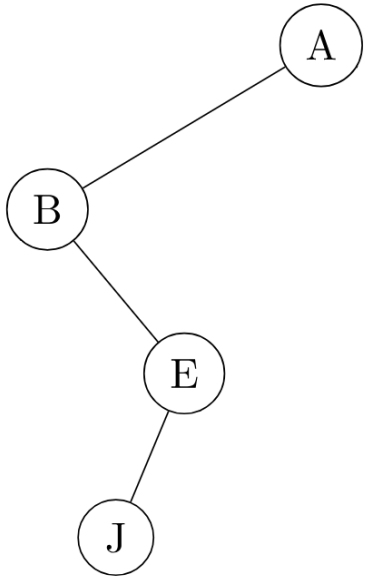




Degenerate Binary Tree

- Behaves like a linked list
- If all the nodes in the degenerate tree have only left child, then it is known as a left-skewed tree





Find function in BST

```
T find(T x) {
  Node *w = r, *z = nil;
  while (w != nil) {
    int comp = compare(x, w->x);
    if (comp < 0) {
      z = w;
      w = w->left;
    } else if (comp > 0) {
      w = w->right;
    } else {
      return w->x;
    }
  }
  return z == nil ? null : z->x;
}
```

Start from the root node

Compare the node value against the given node x value

If comp is <0, the given node x value is < current node value so **we store reference of smallest value >= x in z** and go to left sub-tree

If comp is >0, the given node x value is > current node value so we go to right sub-tree

If comp is =0, the given node x value matches the current node value so we return the current node value

If the search was unsuccessful and we could not find any value smaller than the given node value, we return null otherwise we **return the smallest value >= x which is stored in z**

Example of BST::find

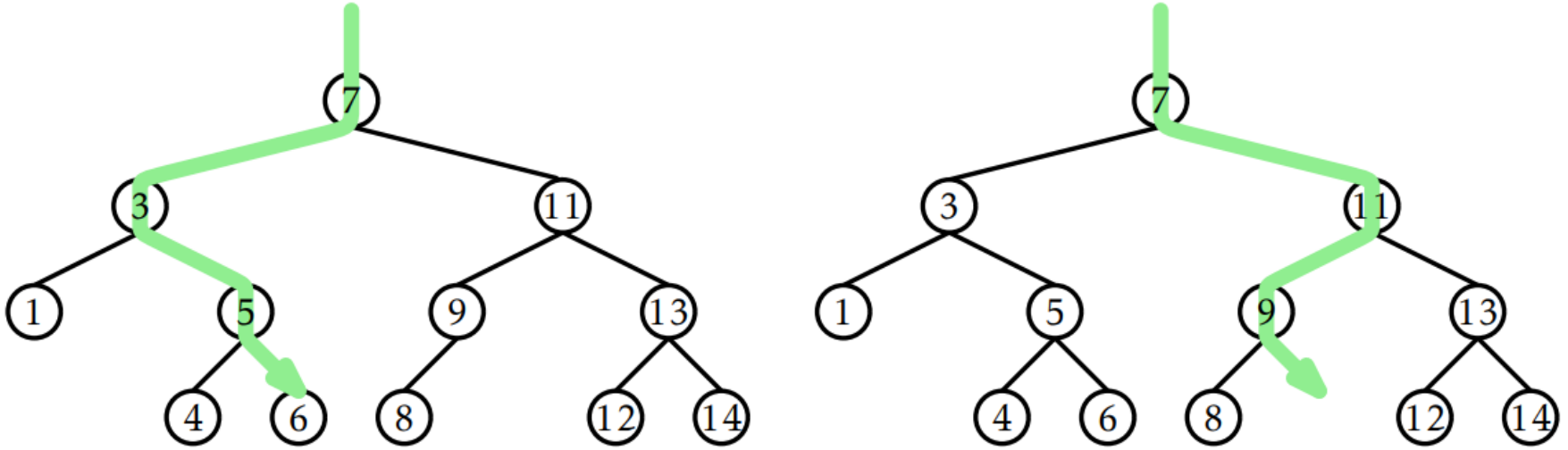


Figure 6.6: An example of (a) a successful search (for 6) and (b) an unsuccessful search (for 10) in a binary search tree.

add function in BST

- It first searches for the given value in the BST
  - If it exists, we don't do anything
  - If it doesn't we store the given value at the node encountered during finding x

```
bool add(T x) {
  Node *p = findLast(x);
  Node *u = new Node;
  u->x = x;
  return addChild(p, u);
}
```

```
Node* findLast(T x) {
  Node *w = r, *prev = nil;
  while (w != nil) {
    prev = w;
    int comp = compare(x, w->x);
    if (comp < 0) {
      w = w->left;
    } else if (comp > 0) {
      w = w->right;
    } else {
      return w;
    }
  }
  return prev;
}
```

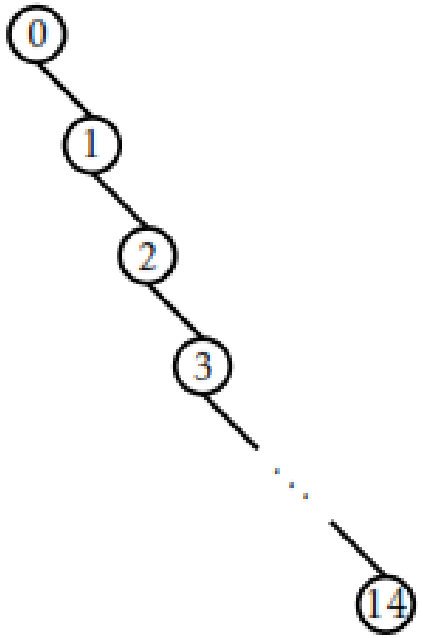
add function in BST


```
bool addChild(Node *p, Node *u) {
    if (p == nil) {
      r = u;                    // inserting into empty tree
    } else {
      int comp = compare(u->x, p->x);
      if (comp < 0) {
        p->left = u;
      } else if (comp > 0) {
        p->right = u;
      } else {
        return false;    // u.x is already in the tree
      }
      u->parent = p;
    }
    n++;
    return true;
  }
```

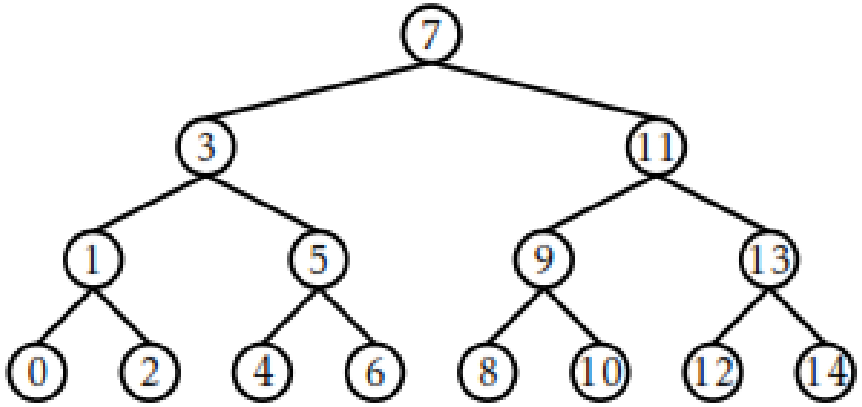
Randomized BSTs

- Same data, two different and valid BSTs

$\langle 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14 \rangle$.



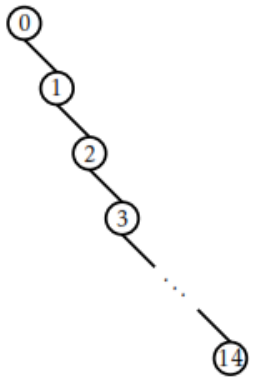
$\langle 7, 3, 11, 1, 5, 9, 13, 0, 2, 4, 6, 8, 10, 12, 14 \rangle$.



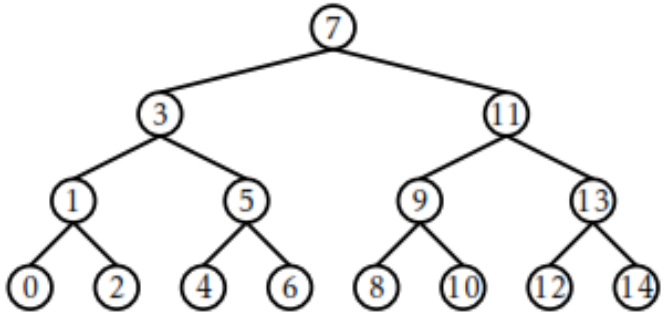
Randomized BSTs

- Same data, two different and valid BSTs

$\langle 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14 \rangle$.



$\langle 7, 3, 11, 1, 5, 9, 13, 0, 2, 4, 6, 8, 10, 12, 14 \rangle$.



- There are 21964800 addition sequences that generate the BST on the right and only one that generates the tree on the left.
  - if we choose a random permutation of 0,...,14, and add it into a binary search tree, then we are more likely to get a very balanced tree (the right side) than we are to get a very unbalanced tree (the left side of Figure 7.1)

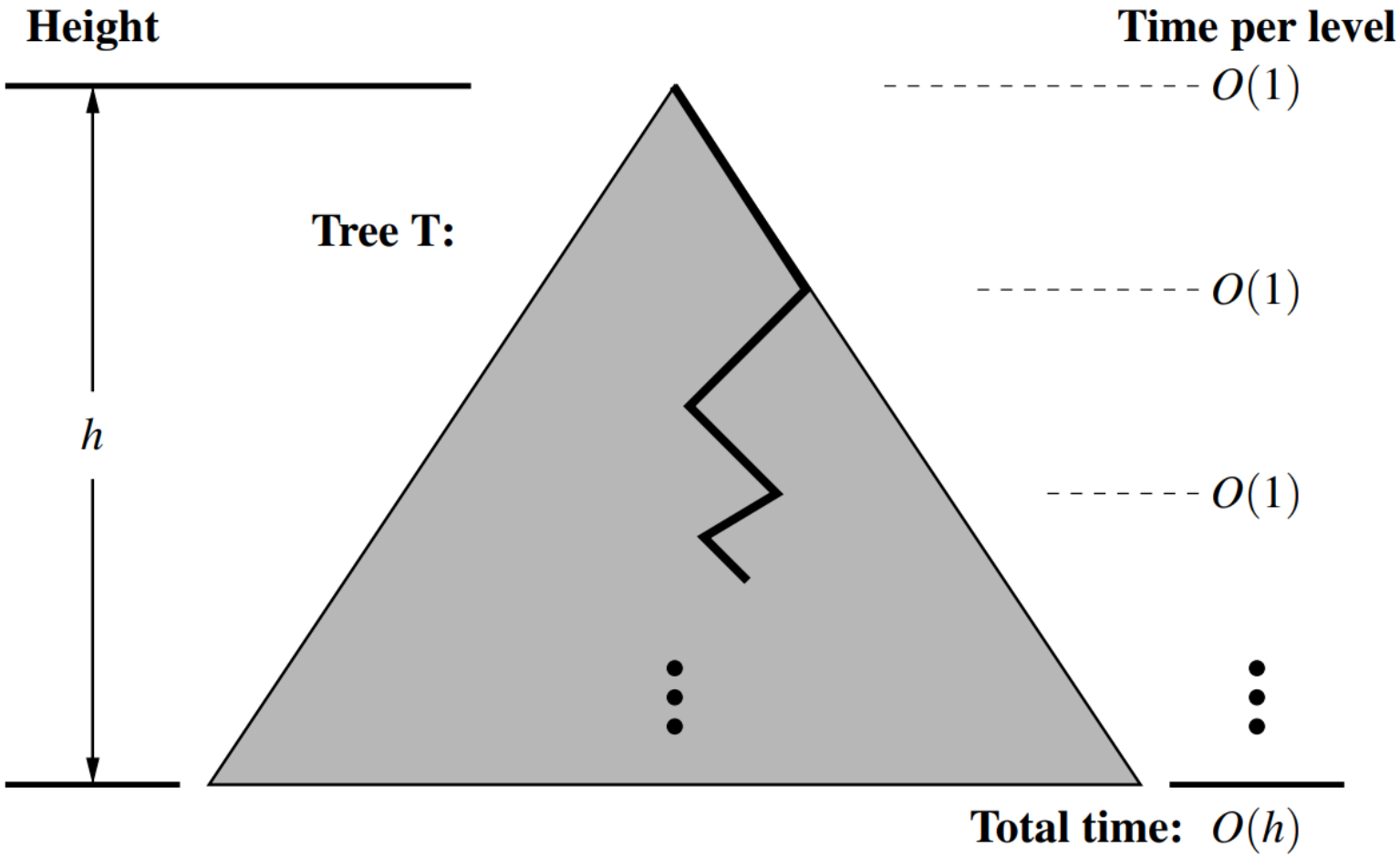
Theorem

Theorem 7.1. A random binary search tree can be constructed in O(nlog(n)) time. In a random binary search tree, the find(x) operation takes O(log(n)) expected time.

Proof:

- To search x in the random BST, we need to find the depth d at which the node with value x appears in the random BST.
- The search starts at the root node and goes down one level each time.
- The depth is bounded by log(n) because the expected height h for a random BST with n elements is log(n)
  - This is because random BST are complete binary tree and expected height of a complete binary tree is log(n)
- Since we spend O(1) expected time per position encountered in the search, the overall search runs in O(h) expected time, where h is the height of the random binary search tree T

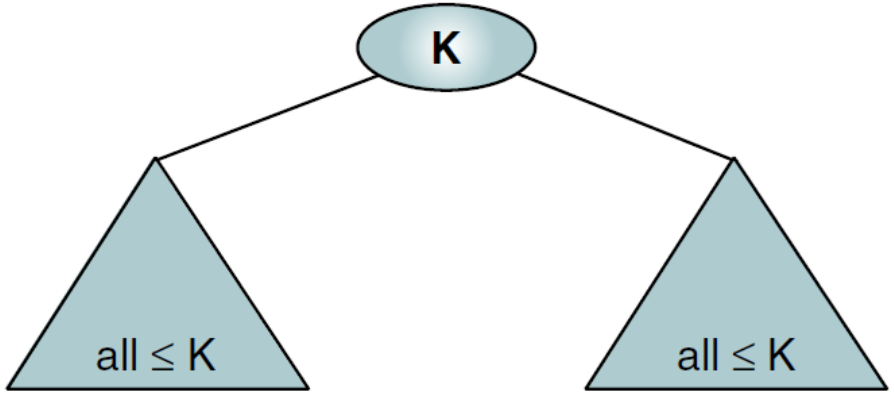
Visualization of proof



What is a heap?

- A heap is a complete or nearly complete binary tree whose left and right subtrees have values
  - less than their parents (max heap)
  - greater than their parents (min heap)

- Unlike BST, the greater value nodes can be placed on either left or right subtree
- Heap property is enforced on both left and right subtrees



Schematic of max heap

Treaps

- Issues with randomized BST
  - they are not dynamic
  - they don't support the add(x) or remove(x) operations needed to implement the SSet interface
- Treap is a blend of tree with heap
- TreapNode is similar to a BSTNode but
  - it contains an additional priority field which is a **unique** numerical value assigned at **random**

```
class TreapNode : public BSTNode<Node, T> {
  friend class Treap<Node,T>;
  int p;
};
```

  - It obeys the heap property that is for every node except the root node, u.parent.p < u.p which means that the root nodes priority is always smallest of its children recursively

Treap structure

- Each treap node contains two fields
  - Key (which follows standard BST ordering left is smaller and right is larger on all subtrees)
  - Priority (randomly assigned priority which follows min heap property that is the root node at all levels contains the least priority of all of its children)

- Why are the heap and binary search conditions combined?
  - They ensure that, once the key (x) and priority (p) for each node are defined, the shape of the Treap is completely determined.

- You can think of a treap as a binary search tree whose nodes are inserted in increasing order of priority

Example of Treap

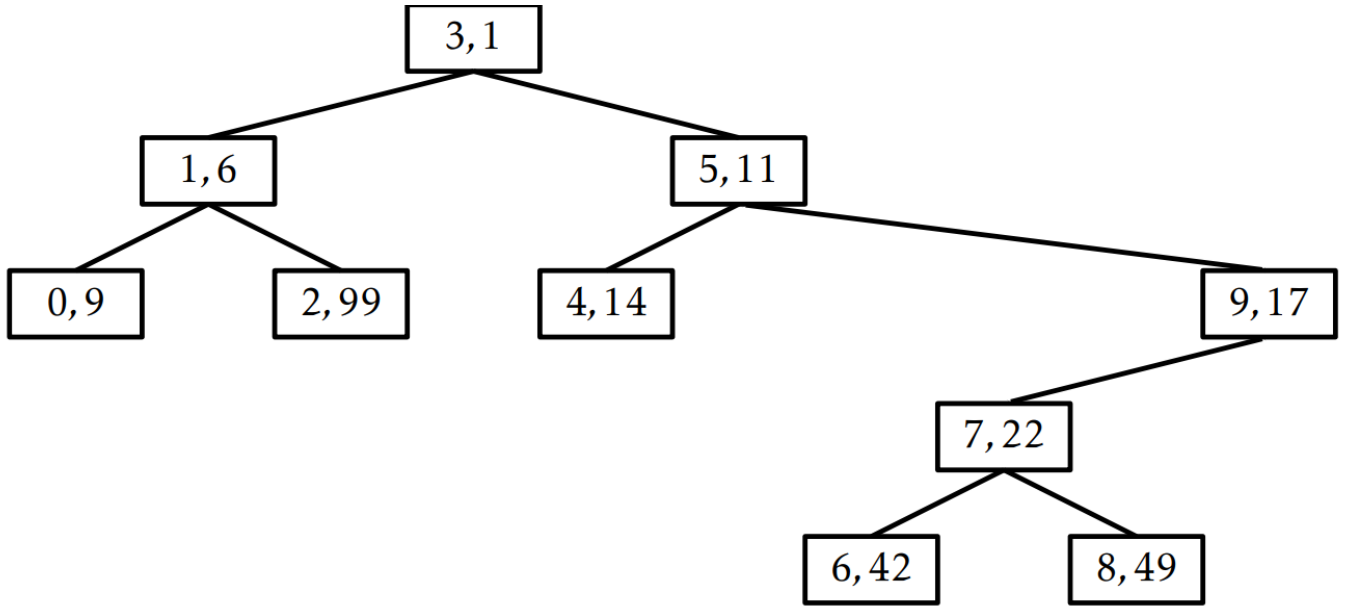


Figure 7.5: An example of a Treap containing the integers $0, \ldots, 9$. Each node, u, is illustrated as a box containing $u.x, u.p$.

Example of Treap

- The previous figure is generated using the following data elements

  (3,1),(1,6),(0,9),(5,11),(4,14),(9,17),(7,22),(6,42),(8,49),(2,99)

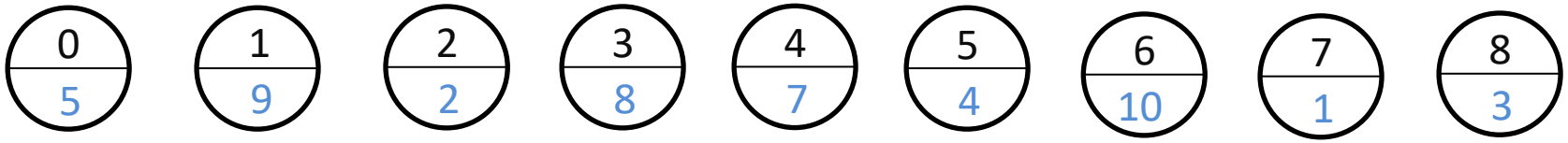
  – Note that the nodes are inserted in ascending order of priority

- Random priority assignment creates a random permutation of the keys before adding them to binary search tree

  [3,1,0,5,9,4,7,6,8,2]

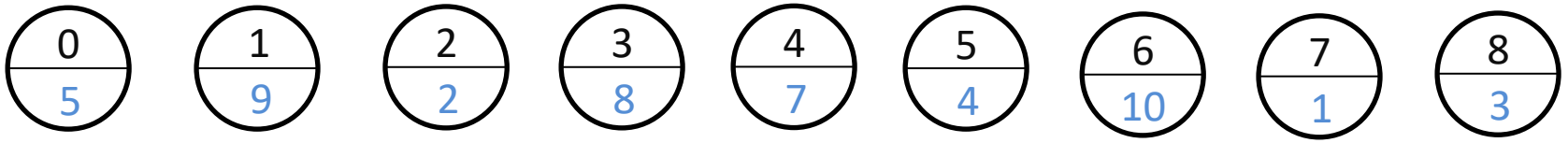
Exercise: Build a Treap

- Keys
- Priorities



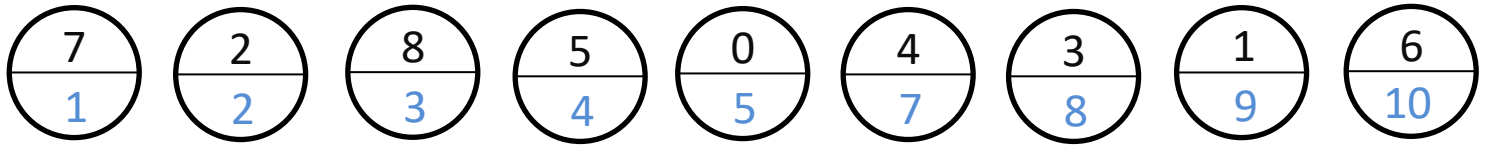
Exercise: Build a Treap

- Keys
- Priorities



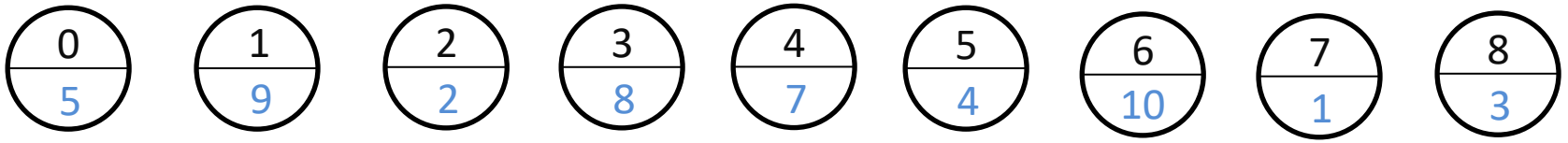
- How to approach the solution?
  - Sort the nodes in ascending order of priority (if priority as minheap)
  - Insert them in BST order

- Sort on priority

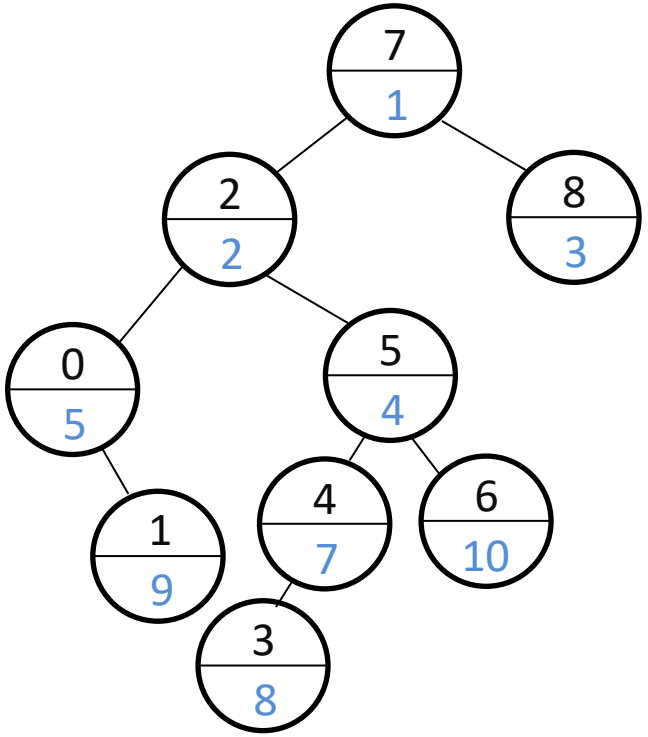


Exercise: Build a Treap

- Keys
- Priorities

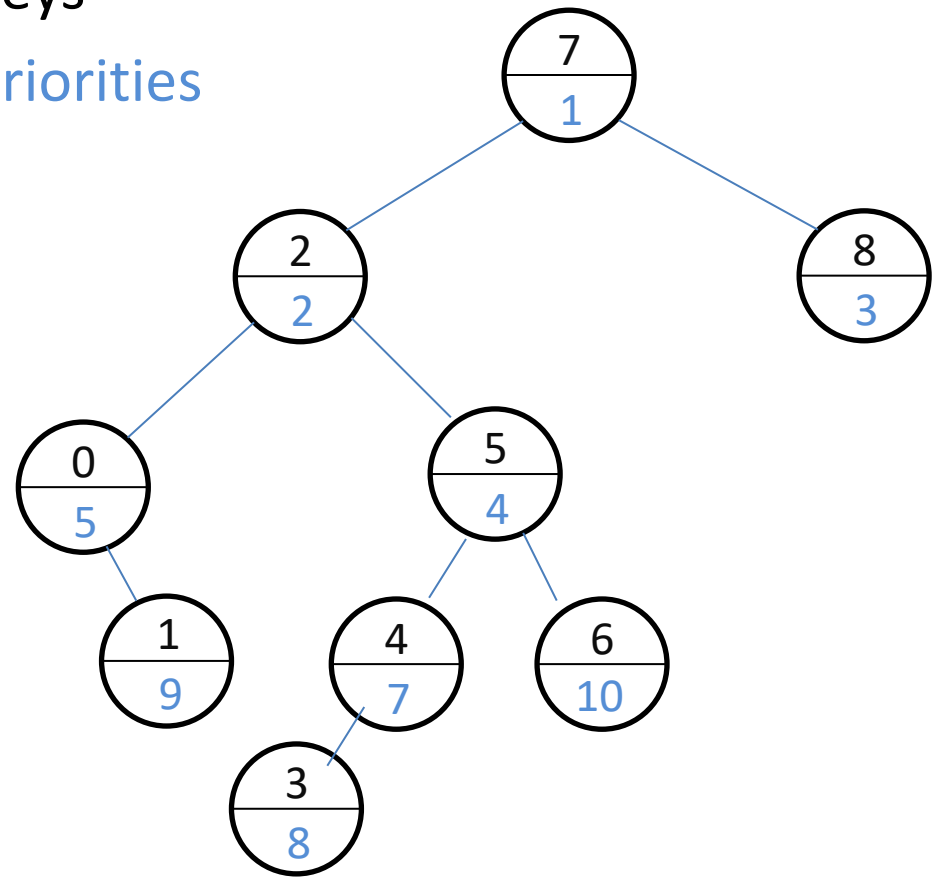


- How to approach the solution?
  - Sort the nodes in ascending order of priority
  - Insert them in BST order

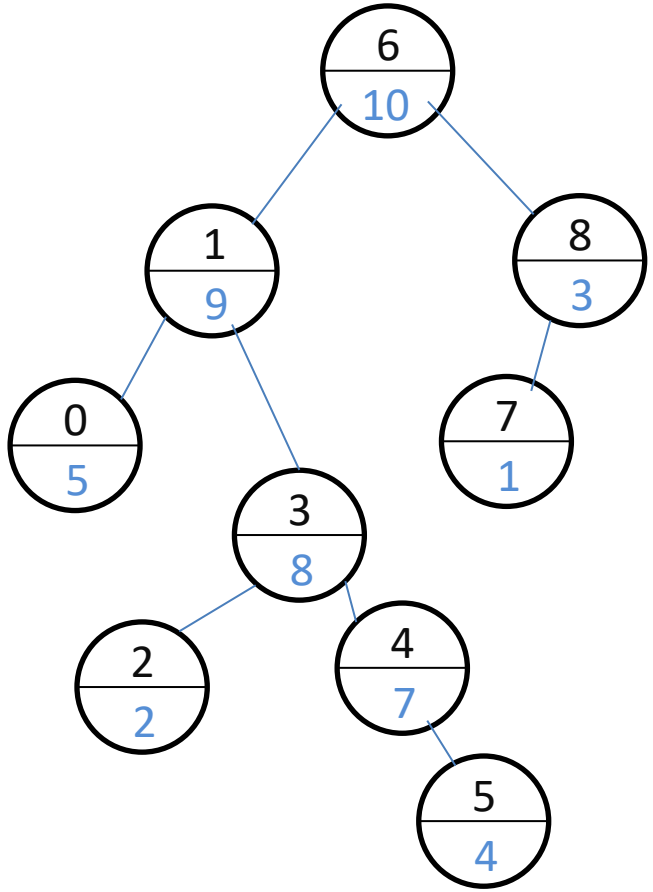


Exercise: Build a Treap - Solution

- Keys
- Priorities



Priority as Minheap



Priority as Maxheap

Treap rotation

- A rotation in a binary search tree is a local modification that takes a parent u of a node w and makes w the parent of u, while preserving the binary search tree property.

- Treaps use rotations to maintain the heap property on priority and the binary search tree property on the key

- Rotations come in two flavours: left or right depending on whether w is a right or left child of u, respectively

Two types of rotations

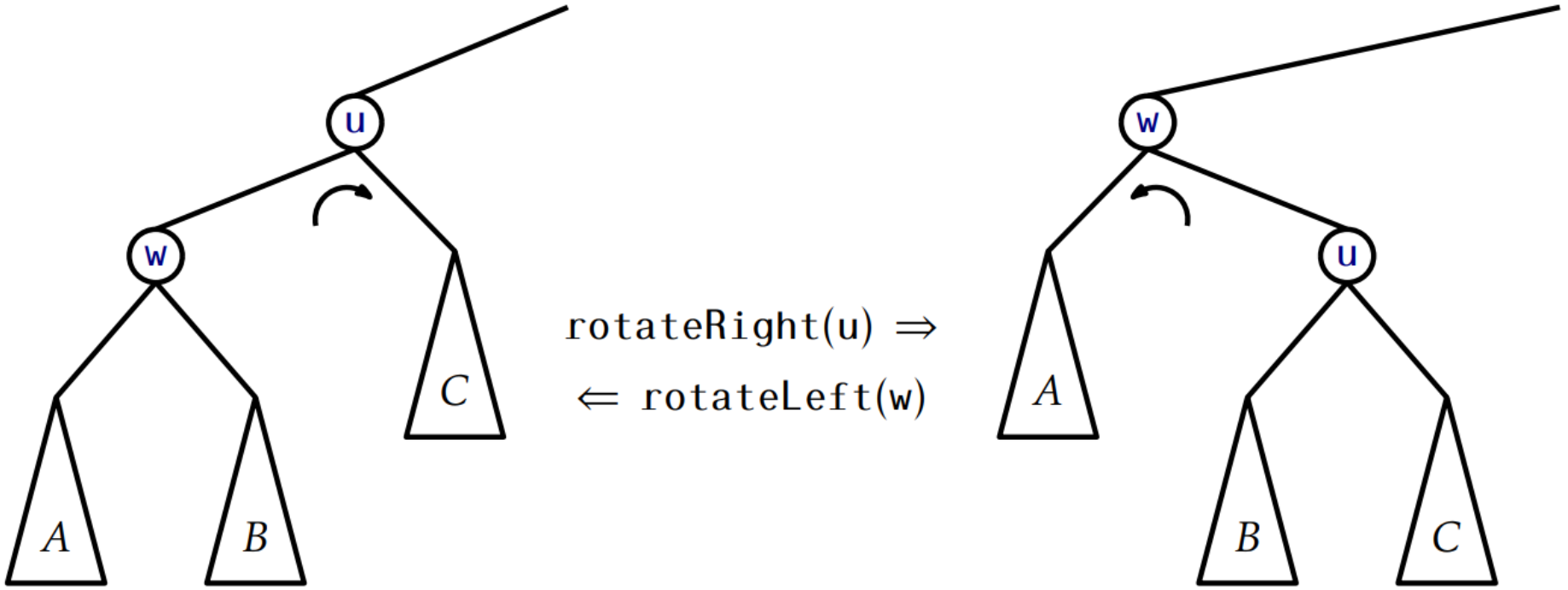
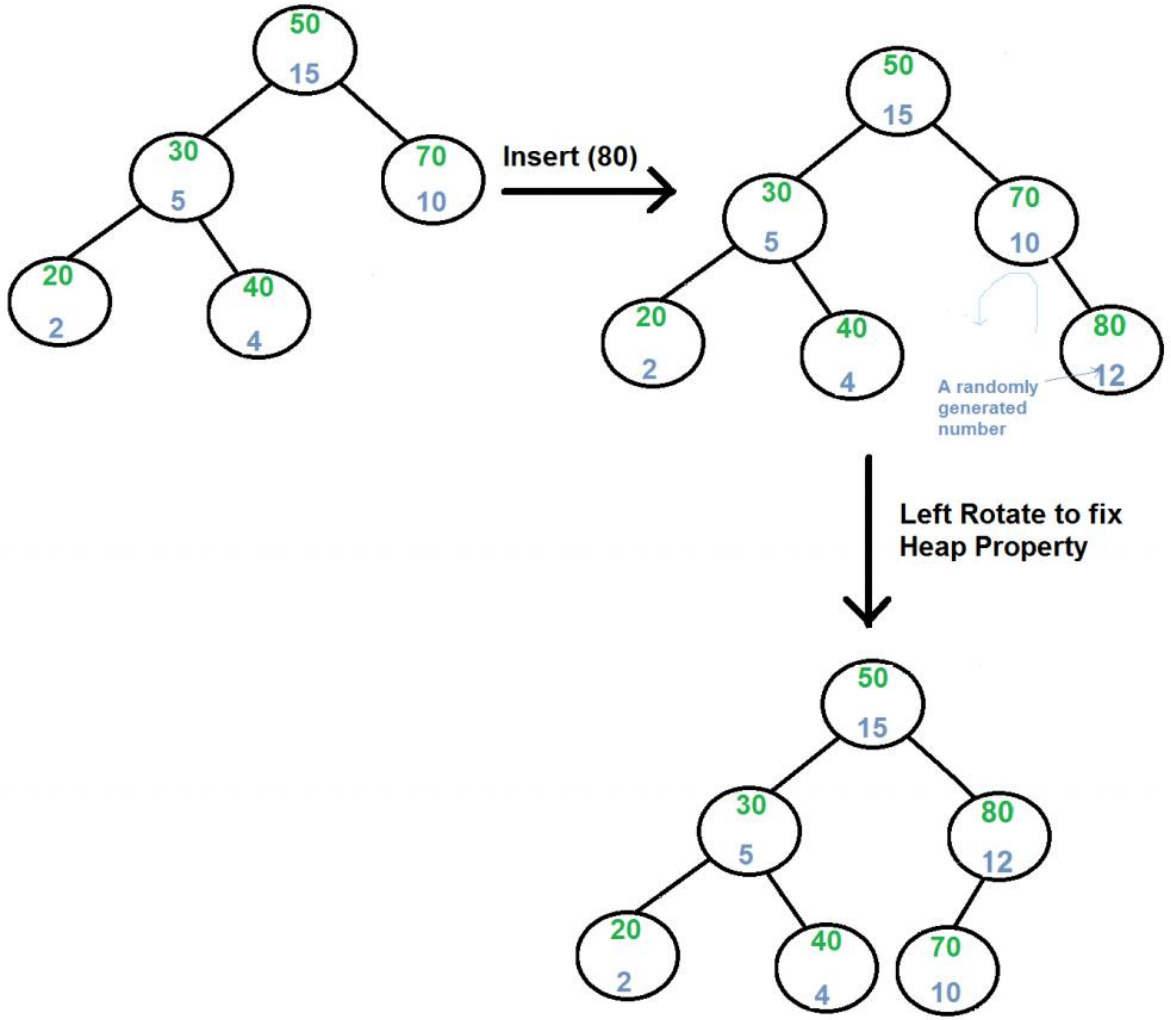


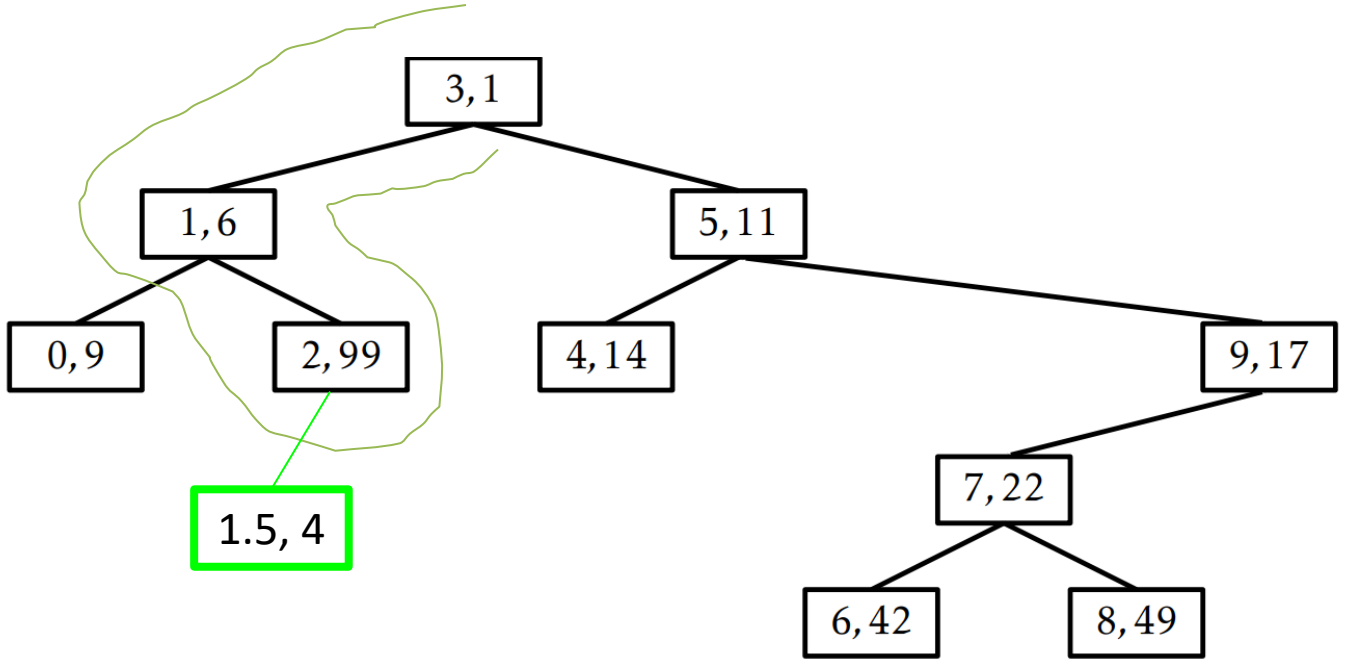
Figure 7.6: Left and right rotations in a binary search tree.

Example of Insertion in Treap



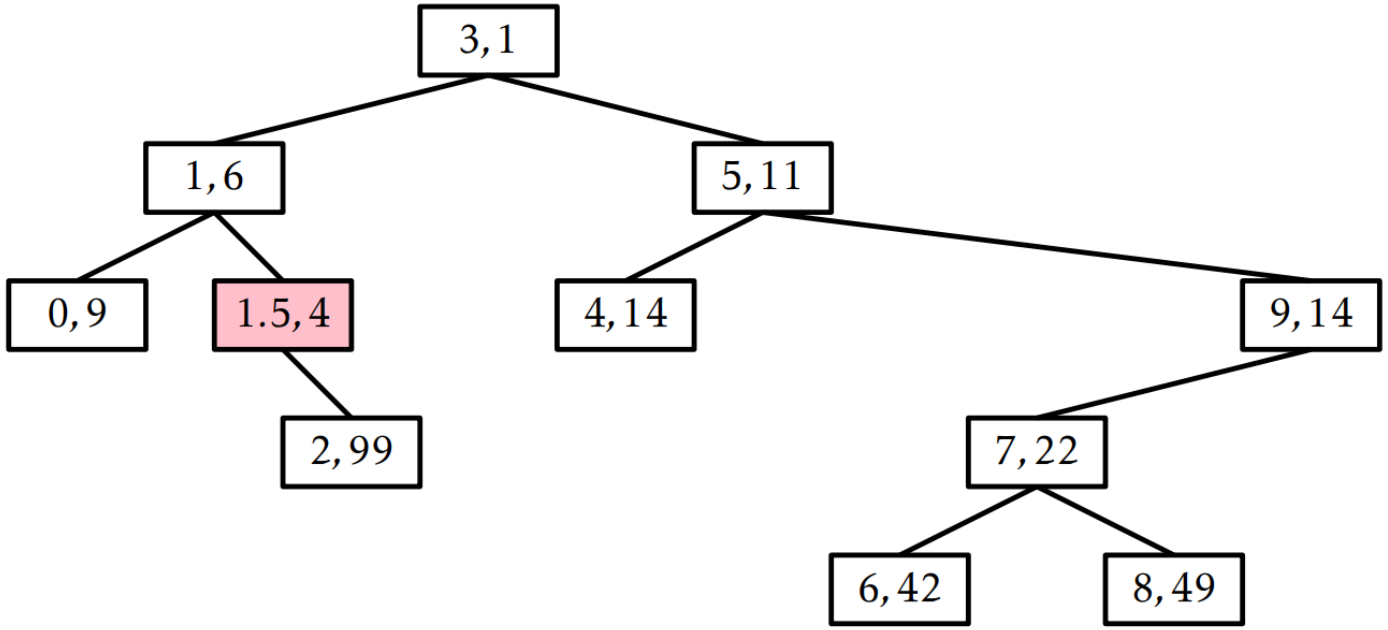
Example of add

- Add u(1.5, 4) key 1.5 with prority 4
  - We use the BST::add function to find the node where the new node may be inserted as a leaf
  - In our example this is node 2,99 in the left sub-tree, node u is inserted to left of node (2,99)



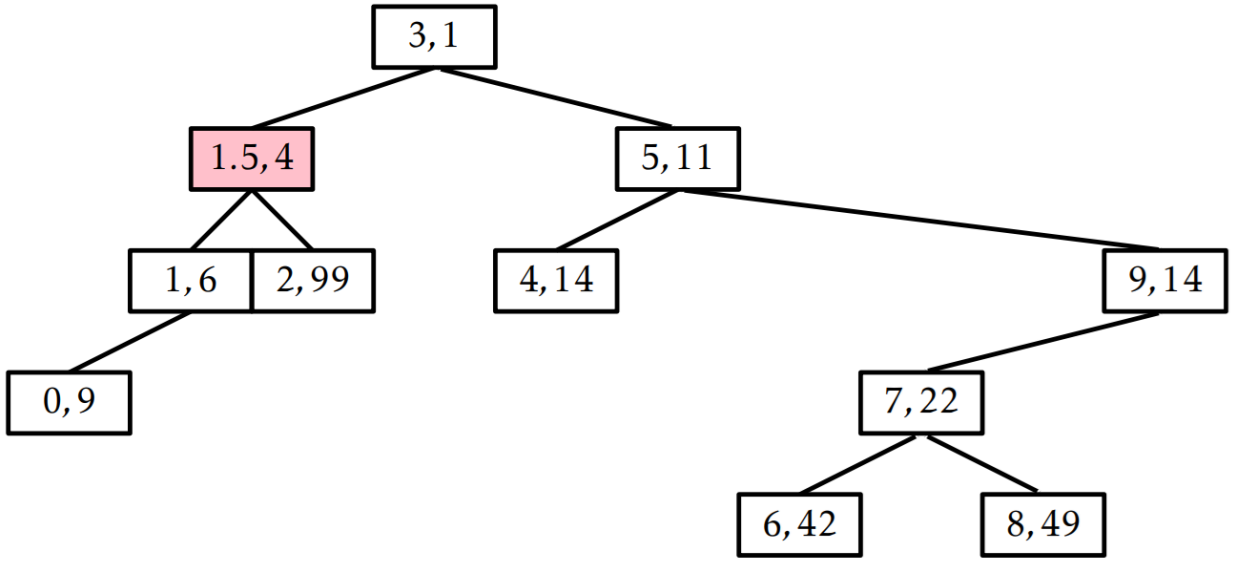
Example of add

- Add u(1.5, 4) key 1.5 with prority 4
  - Next, we notice that the heap property is violated because the node u priority (4) is lower than its parent node's priority (99)
  - To solve this we do right rotation at node (2,99)
  - The right rotation causes the node u to become the parent since it has a lower priority.



Example of add

- Add u(1.5, 4) key 1.5 with prority 4
  - Next, we notice that the heap property is violated again as the parent node (1,6) has a priority of 6 but the child node (1.5,4) has a priority of 4
  - To solve this we do left rotation at node (1.5,4)
  - After left rotation, node u (1.5,4) becomes the root, its previous parent (1,6) becomes its left child

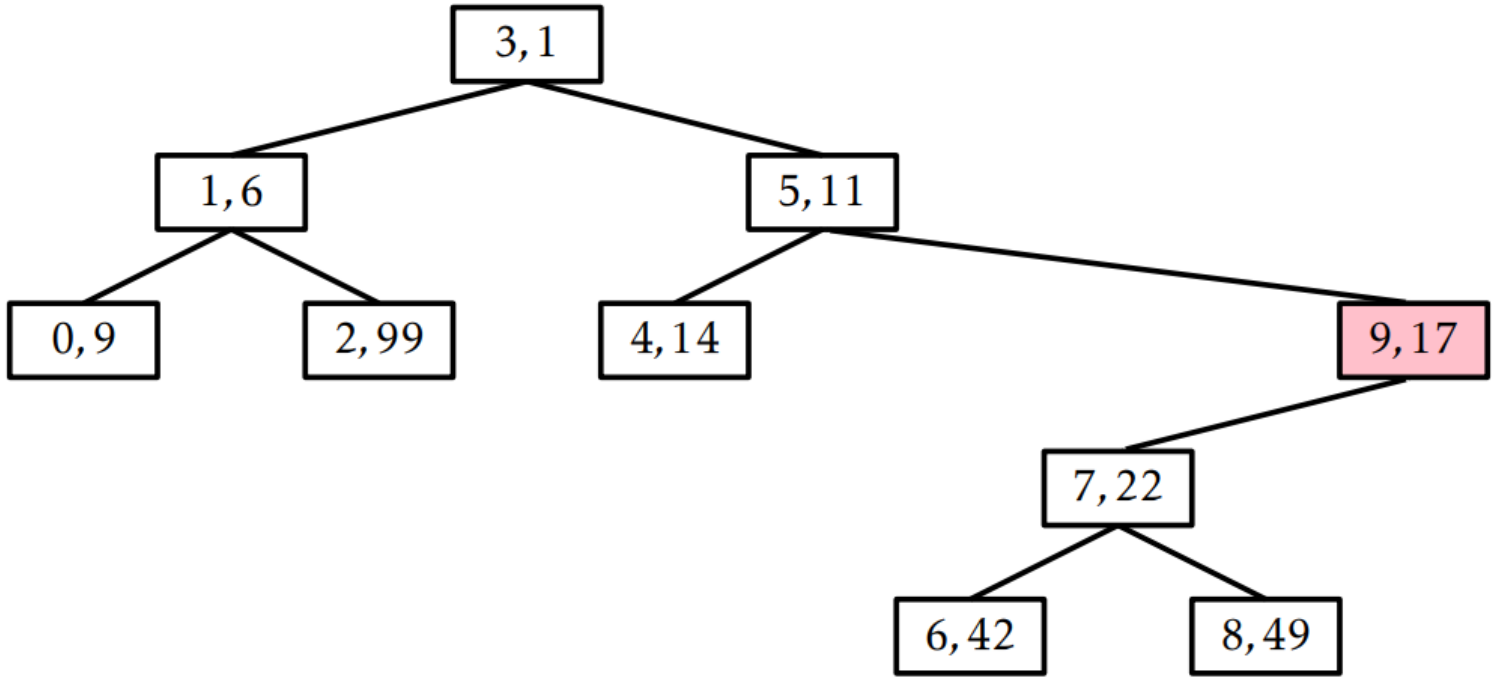


Remove operation in Treap

- Remove proceeds as follows:
  - If node to be deleted is a leaf, delete it.
  - else replace node's priority with minus infinite (-INF), and do appropriate rotations to bring the node down to a leaf and then delete it.

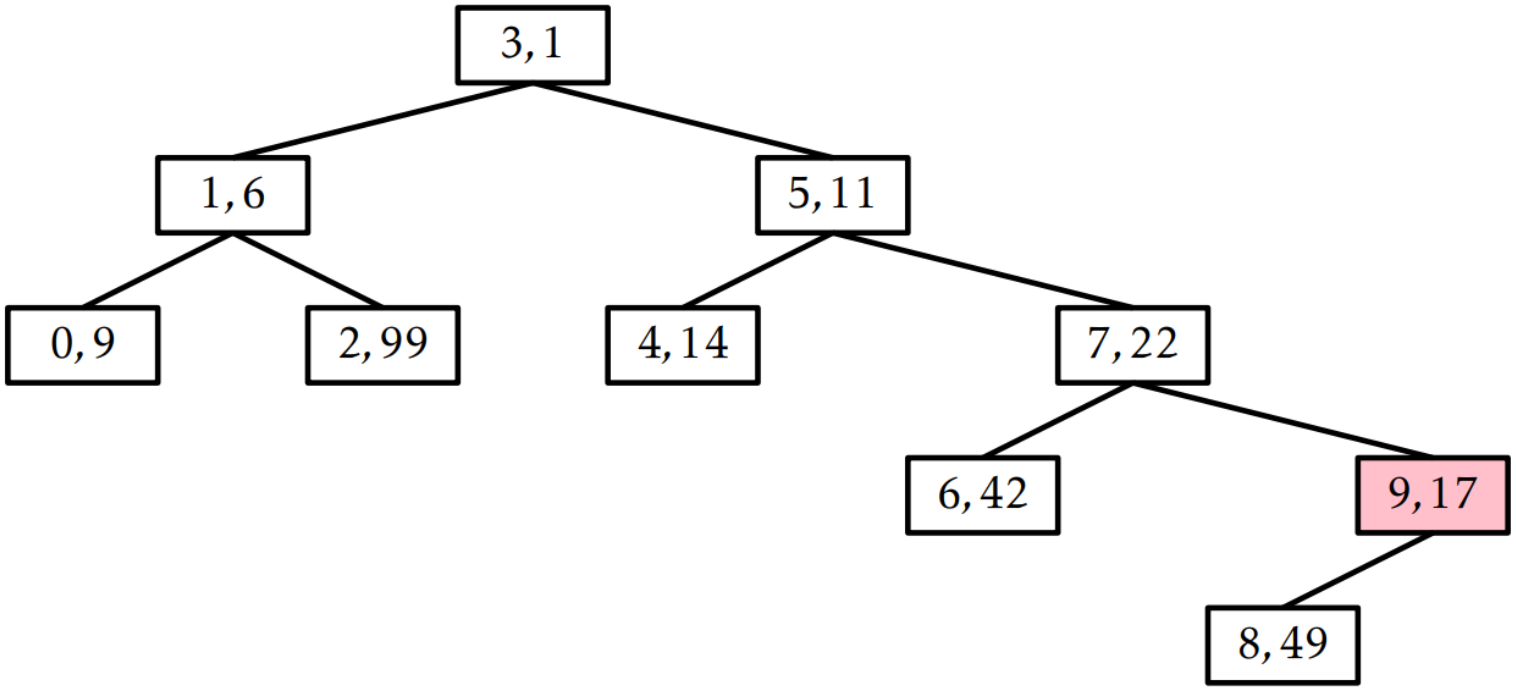
Example of remove

- Lets say we want to remove the node (9,17)
  - We apply right rotation to bring the node (9,17) as leaf



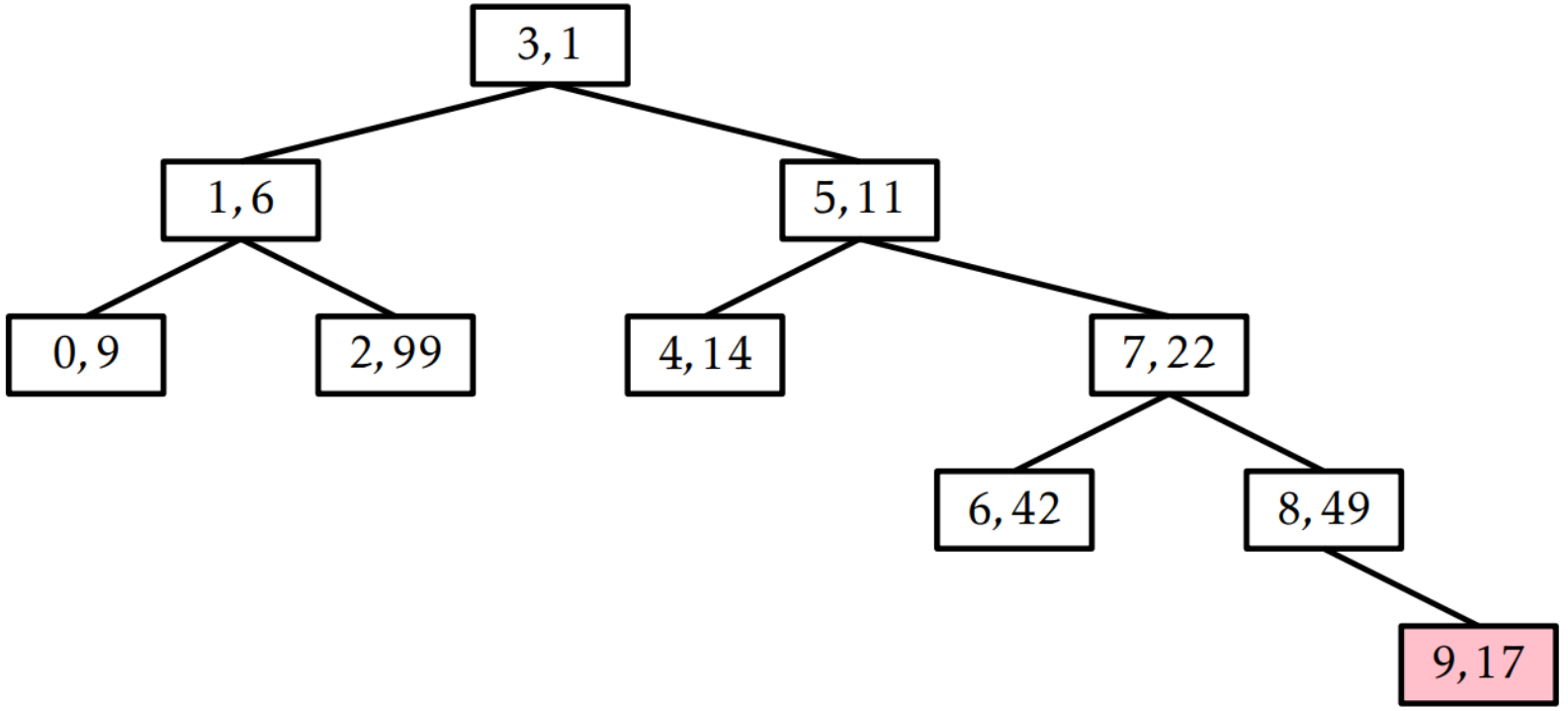
Example of remove

- Lets say we want to remove the node (9,17)
  - Node (7,22) becomes the parent with the node (9,17) becoming its right child. The right child of (7,22) becomes the left child of node (9,17)



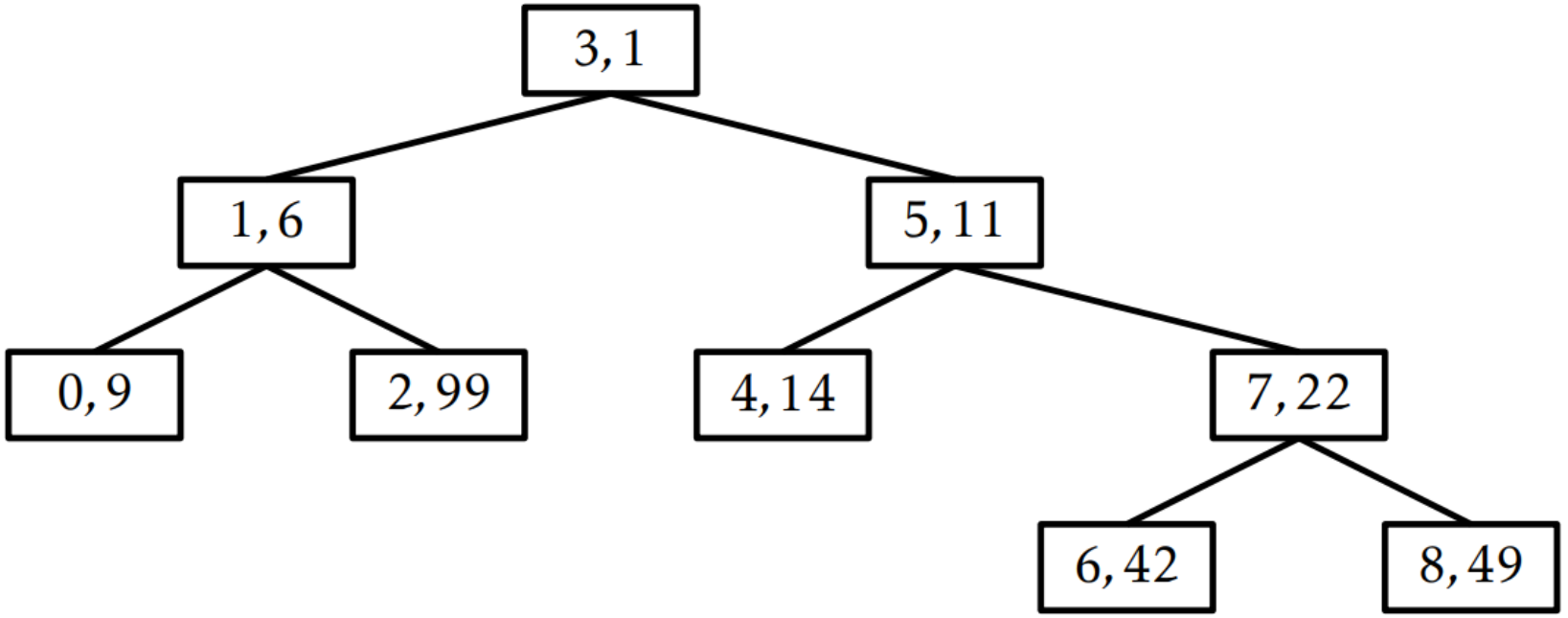
Example of remove

- Lets say we want to remove the node (9,17)
  - We do another right rotation which makes node (8,49) the root and the node (9,17) becomes the leaf.

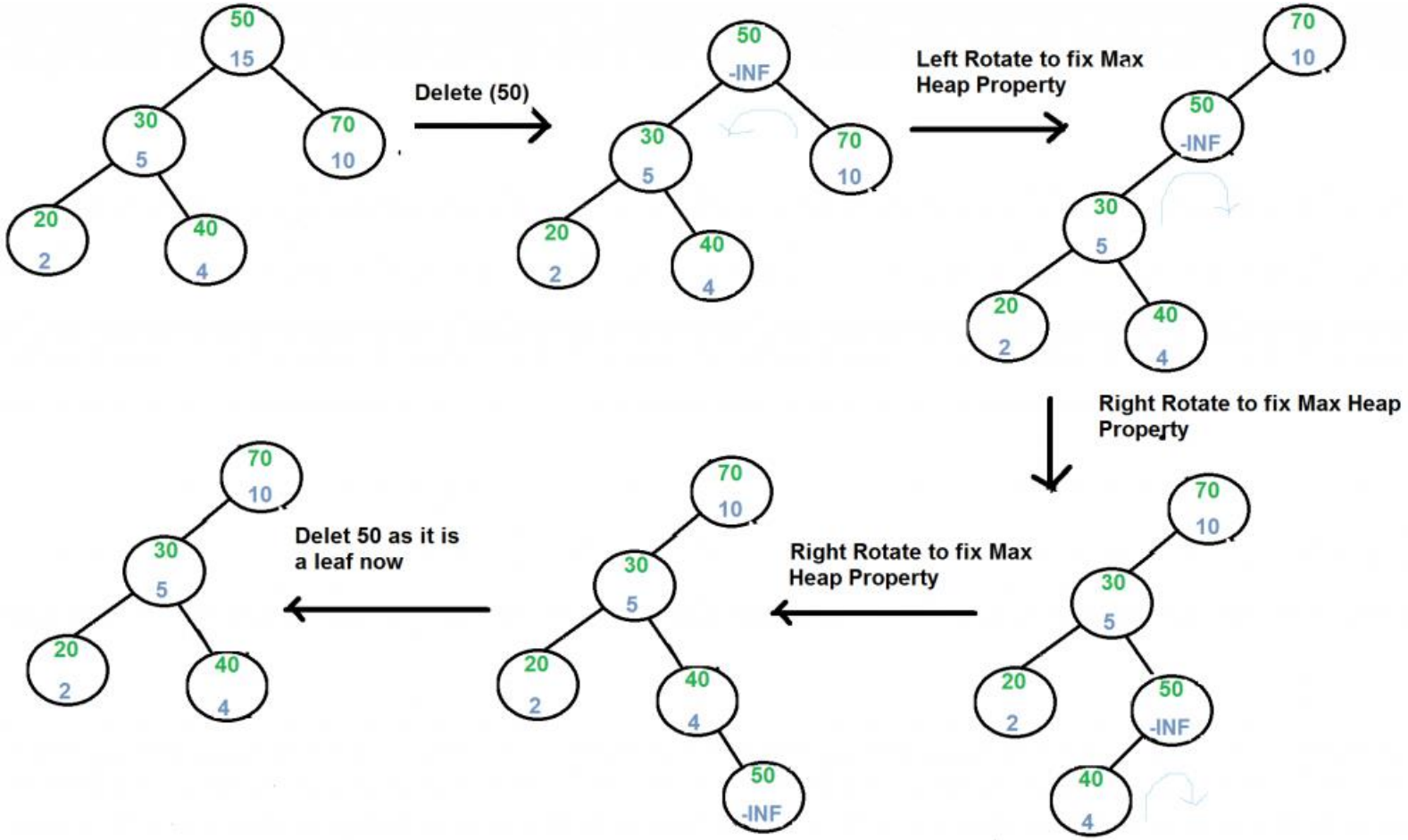


Example of remove

- Lets say we want to remove the node (9,17)
  - Once the node to be deleted becomes the leaf it can be removed



Another example of remove



Comparing Treaps to SkiplistSSet

- Both implement SSet operations in O(log(n)) expected time.
  - add(x) and remove(x) operations in both of these data structures involve a search and then a constant number of pointer changes.
  - Rotations in treaps are O(1) operations

- The expected length of the search path
  - In SkiplistSSet is 2log(n)+O(1)
  - In Treap its 2ln(n)+O(1) ≈ 1.386 logn + O(1) (see Lemma 7.2)
    - Search paths in a Treap are considerably shorter and this translates into noticeably faster operations on Treap

Theorem

Theorem 7.2: A Treap implements the SSet interface. A Treap supports the operations add(x), remove(x), and find(x) in O(logn) expected time per operation.

Proof:

- The running time of add operation is given by the time it takes to follow the search path for x plus the number of rotations performed to move the new node to its correct location.
  - The expected length of the search path is at most 2 lnn + O(1) (see lemma 7.2)
  - The expected number of rotations cannot exceed the expected length of the search path. The expected number of rotations performed during an addition is actually only O(1).
  - Therefore, the expected running time of the add(x) operation in a Treap is O(logn).

Theorem

- The remove operation works in opposite manner to add. It first searches for the node to be deleted. Then it repeatedly applies rotations to bring the node to be deleted to the leaf level where it is then deleted.
  - If we consider the remove in the reverse direction we notice that the number of rotations and the searches we do in remove are the same as the add operation
  - This means that the expected running time of the remove(x) on a Treap of size n is proportional to the expected running time of the add(x) operation on a Treap of size n−1 which is O(log(n)).
- The find operation works exactly as the standard BST find function. It compares the values to be search with the root node value. If the given value is less, we go down the left subtree other wise, we go down the right subtree.
  - Every step, we reduce the number of nodes to be looked up by half, therefore, the BST find function has a expected running time of O(log(n)). The find operation in Treap therefore has the expected running time of O(log(n)).

Book Reading

- ODS
  - Binary Trees, Chapter 6, pages: 133-151
  - Treap, Chapter 7, pages: 153-172

Next Lecture

- Lecture 10